

EJERCICIO ALQUILER DE VEHÍCULOS



Alejandro Bautista Jorge

Introducción.

Vamos a realizar un **ejercicio** en el cual se nos piden tres cosas, el **modelo entidad relación**, el **modelo relacional** y una **base de datos**.

El **modelo entidad relación** ha de ser **extendido** además **normalizado** en **3FN**. Para la realización de dicho ejercicio vamos a utilizar draw.io, para poder realizar el modelo entidad relación.

Para el **modelo relacional** no he necesitado ningún programa, lo he hecho en un bloc de notas.

Para crear la **base de datos** he utilizado [Visual Studio Code](https://code.visualstudio.com/) como IDE, es decir, para hacer el código y luego además [XAMPP](https://www.apachexampproject.org/) para correr el servidor [MySQL](https://www.mysql.com/) y poder probar el código.

Modelo E/R.

El modelo E-R está justificado porque representa de forma precisa las entidades y relaciones necesarias en un sistema de alquiler y reparación de vehículos, diferenciando correctamente a los clientes, vehículos, alquileres, reparaciones y piezas.

El diseño cumple la **Tercera Forma Normal (3FN)** porque cada entidad tiene una clave primaria bien definida, todos los atributos dependen únicamente de dicha clave, no existen dependencias transitivas ni redundancias, y las relaciones N:N se han descompuesto mediante tablas intermedias.

Además, el modelo es **extendido** porque incorpora una **generalización** del vehículo en distintos tipos (Turismo y Furgoneta), lo que permite añadir atributos específicos a cada categoría sin duplicar información y aporta una estructura más flexible y detallada del dominio.

Modelo relacional.

El modelo relacional refleja correctamente la estructura del sistema, usando claves primarias para identificar cada registro y claves foráneas que garantizan la integridad entre tablas. **TURISMO** y **FURGONETA** derivan de **VEHÍCULO** mediante una generalización, heredando la clave sin duplicar información. **ALQUILER** y **REPARACION** vinculan vehículos y clientes según las necesidades del negocio. La relación N:N entre reparaciones y piezas se resuelve mediante **REPARACION_PIEZA**, con clave compuesta y atributo de cantidad.

El diseño está en **3FN**, ya que todos los atributos dependen solo de la clave primaria de su tabla, no hay dependencias transitivas y se evita la redundancia, garantizando un modelo coherente y eficiente para su implementación.

Base de datos (SQL).

La base de datos está diseñada para un **sistema de gestión de alquiler de vehículos**. La tabla **Vehiculo** almacena la información común de todos los vehículos, mientras que **Turismo** y **Furgoneta** contienen atributos específicos y se relacionan mediante **claves foráneas** con eliminación y actualización en cascada. La tabla **Cliente** guarda los datos de los clientes, y **Alquiler** vincula clientes y vehículos, registrando fechas y duración del alquiler. Para el mantenimiento, **Pieza** y **Reparacion** registran piezas y reparaciones, y **Reparacion_Pieza** establece la relación **muchos a muchos** entre ellas. Este diseño asegura **integridad de los datos**, evita duplicados y facilita consultas sobre vehículos, clientes, alquileres y reparaciones.